

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 10%

QUESTIONS: 15

Total Marks:20

Annexure A

Comparative Analysis

Code of Conduct:

I Jeevan Raaja S (192421005) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

<i>Criteria</i>	<i>Conceptual Understanding</i>	<i>Accuracy of Answers</i>	<i>Application of Knowledge</i>	<i>Completeness of Response</i>	<i>Clarity & Presentation</i>	<i>Total</i>
<i>Weightage</i>	<i>30%</i>	<i>15%</i>	<i>20%</i>	<i>20%</i>	<i>15%</i>	
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						

Signature of the student:

Total Marks :

Annexure A

Short Answer Test

1. Compare **Linear Regression and Polynomial Regression** for predicting housing prices. Analyze differences in model complexity, ability to capture non-linear relationships, risk of overfitting, and interpretability. Discuss scenarios where each model performs better.
2. Compare **Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM)** networks for time-series prediction tasks such as stock price forecasting. Analyze differences in handling long-term dependencies, vanishing gradient problem, computational complexity, and prediction accuracy.
3. Compare **Naïve Bayes and Support Vector Machines (SVM)** for classification problems such as sentiment analysis. Analyze differences in assumptions, decision boundaries, computational efficiency, and performance on high-dimensional data. Discuss scenarios where each method is preferable.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q1. Linear Regression vs Polynomial Regression for Housing Price Prediction

Introduction

Both **Linear Regression** and **Polynomial Regression** can be used to predict housing prices. The main difference is that Linear Regression models a straight-line relationship, while Polynomial Regression can model curved and more complex relationships between input features and house prices.

Comparison

Factor	Linear Regression	Polynomial Regression
Model	Straight-line relationship	Curved/non-linear relationship
Complexity	Low	Higher
Non-linear relationships	Cannot capture complex curves effectively	Can capture non-linear patterns
Overfitting risk	Low	Higher, especially with high degree
Interpretability	Easy to interpret	More difficult
Computation	Fast	More computationally expensive
Data requirement	Works well with simpler relationships	Requires sufficient data for complex patterns

Linear Regression

Linear Regression predicts the housing price using a linear equation:

For example, if house price mainly increases consistently with **house area**, Linear Regression can provide a simple and understandable prediction.

Advantages:

- Simple to implement.
- Fast to train.
- Easy to interpret.
- Lower risk of overfitting.

Best scenario:

It performs well when the relationship between house features and price is approximately linear.

Polynomial Regression

Polynomial Regression extends Linear Regression by adding powers of the input variables.

For example:

It can capture curved relationships between house characteristics and price.

For example, housing prices may not increase at the same rate for every increase in house area. Polynomial Regression can model such non-linear behaviour.

Advantages:

- Captures non-linear relationships.
- Can provide better predictions when the data contains curves.
- More flexible than simple Linear Regression.

Disadvantages:

- More complex.
- Higher-degree polynomials can overfit.
- More difficult to interpret.

Overfitting

Overfitting occurs when a model learns the training data too closely and performs poorly on unseen data.

Linear Regression generally has a lower risk of overfitting because of its simple structure.

Polynomial Regression has a higher risk, especially when a very high polynomial degree is selected.

Therefore, the polynomial degree should be selected carefully using validation techniques.

Practical Application

For housing price prediction:

Linear Regression is preferable when:

- House price has a roughly linear relationship with area.
- The dataset is relatively simple.
- Interpretability is important.
- Fast prediction is required.

Polynomial Regression is preferable when:

- Housing prices show a clear non-linear relationship with features.
- A straight line cannot represent the data accurately.
- The dataset contains enough observations to support a more complex model.

Conclusion

Linear Regression is **simple, fast, and highly interpretable**, making it suitable for housing datasets with approximately linear relationships. Polynomial Regression is **more flexible** and can capture non-linear relationships, but it has greater complexity and a higher risk of overfitting. Therefore, the choice depends on the relationship present in the housing data.

Q2. RNN vs LSTM for Time-Series Prediction

The question asks you to compare **Recurrent Neural Networks (RNNs)** and **Long Short-Term Memory (LSTM)** networks for time-series prediction such as stock-price forecasting, focusing on long-term dependencies, vanishing gradients, computational complexity, and prediction accuracy.

Introduction

RNN and LSTM are neural-network architectures designed to process **sequential data**. They can use previous observations to predict future values. For example, in stock-price forecasting, previous prices and other time-dependent information can be used to predict future prices.

Comparison

Factor	RNN	LSTM
Long-term dependencies	Difficult to handle	Handles long-term dependencies effectively
Vanishing gradient	More affected	Designed to reduce this problem
Architecture	Simple	More complex

Factor	RNN	LSTM
Computational complexity	Lower	Higher
Training speed	Generally faster	Generally slower
Memory	Limited for long sequences	Better memory of important information
Prediction accuracy	Good for short-term patterns	Generally better for long-term patterns

Recurrent Neural Network (RNN)

An RNN processes sequential information by maintaining a **hidden state** that carries information from previous time steps.

For example:

Previous stock prices → RNN → Hidden state → Future price prediction

RNNs are useful when recent observations have a strong influence on the current prediction.

However, standard RNNs can have difficulty remembering information from much earlier time steps.

Long Short-Term Memory (LSTM)

LSTM is an improved type of recurrent neural network designed to handle **long-term dependencies**.

It uses a memory cell and gates to control information:

- **Forget gate** – decides what information should be removed.
- **Input gate** – decides what new information should be stored.
- **Output gate** – decides what information should be passed to the next stage.

This allows LSTM to retain important information over longer sequences.

Long-Term Dependencies

For stock-price forecasting, information from several earlier time periods may sometimes be useful.

A standard RNN may gradually lose older information as the sequence becomes longer.

LSTM is better suited for this situation because its memory mechanism allows important information to be retained for longer periods.

Vanishing Gradient Problem

During training, standard RNNs can suffer from the **vanishing gradient problem**. Gradients can become extremely small as they are propagated through many time steps.

As a result, the network has difficulty learning relationships between distant events.

LSTM reduces this problem through its memory-cell and gate mechanism, allowing important information and gradients to be carried across longer sequences.

Computational Complexity

RNNs have a simpler architecture and therefore generally require **less computation and fewer parameters**.

LSTMs contain multiple gates and a memory cell, making them more computationally expensive and generally slower to train.

Therefore:

RNN → simpler and faster

LSTM → more complex but better memory

Prediction Accuracy

RNNs can perform well when the prediction mainly depends on **recent observations** and the sequence is relatively short.

LSTMs are generally more suitable when the data contains **long-term dependencies**.

For stock-price forecasting, the final accuracy depends on the dataset, features, preprocessing, sequence length, and model configuration. Therefore, LSTM should not automatically be assumed to be more accurate for every dataset.

Practical Application

RNN is preferable when:

- The sequence is relatively short.
- Recent observations are more important.
- A simpler model is required.
- Lower computational cost is preferred.

LSTM is preferable when:

- Long-term dependencies are important.
- The sequence is long.
- Previous information from distant time steps may influence predictions.
- The vanishing-gradient problem is a concern.

Conclusion

RNN and LSTM are both useful for time-series prediction. **RNNs are simpler and computationally less expensive**, but they have difficulty learning long-term dependencies because of the vanishing-gradient problem. **LSTMs use memory cells and gates to retain important information for longer periods**, making them more suitable for complex time-series tasks such as stock-price forecasting when long-term dependencies are important.

Q3. Naïve Bayes vs SVM for Sentiment Analysis

The question asks you to compare **Naïve Bayes** and **Support Vector Machines (SVM)** for classification tasks such as sentiment analysis, focusing on assumptions, decision boundaries, computational efficiency, high-dimensional data, and suitable use cases.

Introduction

Naïve Bayes and SVM are supervised machine learning algorithms that can be used to classify text into categories such as:

- **Positive sentiment**
- **Negative sentiment**

For sentiment analysis, text is first converted into numerical features using methods such as word counts or TF-IDF. The classifier then learns patterns from labelled training data.

Comparison

Factor	Naïve Bayes	SVM
Basic approach	Probabilistic classifier	Margin-based classifier

Factor	Naïve Bayes	SVM
Main assumption	Features are conditionally independent	Does not require feature independence
Decision boundary	Based on class probabilities	Finds an optimal separating hyperplane
Training speed	Generally very fast	Generally slower than Naïve Bayes
High-dimensional data	Performs well	Performs very well
Memory requirement	Relatively low	Can be higher
Interpretability	Relatively simple	More complex
Sentiment analysis	Very suitable	Very suitable

Naïve Bayes

Naïve Bayes is based on **Bayes' theorem** and calculates the probability that a document belongs to each class.

For example:

The class with the highest probability is selected as the prediction.

The algorithm assumes that the features are **conditionally independent given the class**. Although this assumption is often unrealistic for natural language, Naïve Bayes can still perform well in text classification.

SVM

SVM classifies data by finding an optimal **decision boundary**, or hyperplane, that separates different classes.

For sentiment analysis, the model attempts to separate positive and negative documents using their numerical feature representations.

The SVM tries to maximize the **margin** between the classes. The training points closest to the decision boundary are called **support vectors**.

Assumptions

Naïve Bayes:

The main assumption is that features are conditionally independent.

For example, it treats the presence of different words as independent given the sentiment class.

SVM:

SVM does not make the same feature-independence assumption. It focuses on finding a decision boundary that separates the classes effectively.

Decision Boundaries

Naïve Bayes makes predictions based on **probability estimates** for each class.

SVM directly learns a **decision boundary** that separates classes.

For example:

Positive reviews | Decision boundary | Negative reviews

SVM chooses the boundary that provides the largest possible margin between the classes.

Computational Efficiency

Naïve Bayes is generally **fast to train and predict**, making it useful when large amounts of text must be processed quickly.

SVM can require more computational resources, particularly when the dataset is large or when complex kernels are used. However, a linear SVM is commonly suitable for large text-feature spaces.

High-Dimensional Data

Sentiment-analysis datasets can contain thousands or millions of possible words, resulting in a **high-dimensional feature space**.

Both algorithms can work effectively with high-dimensional text data.

Naïve Bayes is attractive because of its simplicity and speed.

SVM is also well suited because it can find effective separating boundaries in high-dimensional feature spaces.

When Naïve Bayes Is Preferable

Naïve Bayes is preferable when:

- Fast training is required.
- The dataset is large.
- A simple model is desired.
- Quick baseline sentiment classification is needed.
- Computational resources are limited.

When SVM Is Preferable

SVM is preferable when:

- Classification accuracy is important.
- The data contains high-dimensional features.
- A strong separating boundary is required.
- The dataset is suitable for margin-based classification.
- A linear SVM can efficiently handle large text-feature spaces.

Conclusion

Both **Naïve Bayes and SVM are effective algorithms for sentiment analysis**. Naïve Bayes is a probabilistic method that assumes conditional independence between features and is generally fast and simple. SVM does not require this independence assumption and learns an optimal decision boundary by maximizing the margin.

Therefore, **Naïve Bayes is a good choice when speed, simplicity, and computational efficiency are priorities**, while **SVM is preferable when a strong decision boundary and high-dimensional classification performance are important**.